

Asset Management App — Project Structure Guidelines

Sep 26, 2026 · @BigPhu

Overview

Phase 1 is a modular monolith in Go with four modules: identity (login, JWT, RBAC), directory (members, departments, locations), inventory (assets, with Excel/CSV import and export) and activity (the audit trail). It follows clean architecture: inside a module, calls flow from an entry point (handler, worker or subscriber) to service to repository, with shared domain types; between modules, calls go through each module's contract interfaces and published events; events are delivered with the transactional outbox pattern on River; at the HTTP edge, an OpenAPI spec per module is the contract with the frontend.

The app serves a single company, so there is no tenant ID anywhere. The goal is a structure that is easy to follow for anyone joining the project: few layers, one direction of calls, no abstractions added before they are needed.

Concern	Choice
Language	Go, latest stable release
HTTP router	go-chi/chi v5
HTTP contract	OpenAPI 3 spec per module; oapi-codegen generates types, a strict server interface and chi routes
Frontend client	Generated from the same specs (for example openapi-typescript)
Database	PostgreSQL
DB driver	jackc/pgx v5 (pgxpool)
Query generation	sqlc, with sql_package: pgx/v5
Migrations	goose library, SQL files embedded in cmd/migrate ; sqlc reads the same files as schema

Concern	Choice
Background jobs	riverqueue/river (jobs stored in Postgres), also used as the transactional outbox for domain events, run by <code>cmd/worker</code>
Excel	xuri/excelize v2 for .xlsx; <code>encoding/csv</code> for CSV
Password hashing	bcrypt via golang.org/x/crypto/bcrypt
Authentication	JWT access tokens (golang-jwt/jwt v5, HS256, 15-minute expiry) sent as <code>Authorization: Bearer</code>
Architecture checks	golangci-lint with depguard rules for module boundaries
Config	Environment variables parsed into one typed struct at startup

Repository layout

The repository has one API binary, one migrations folder, one sqlc config, and all application code under `internal/`.

```

.
├── cmd/
│   ├── api/main.go           # HTTP server: build modules + event registry,
│   │   │   mount chi, serve
│   ├── worker/main.go       # River: event subscribers, background jobs,
│   │   │   scheduled jobs
│   └── migrate/main.go      # apply goose + River migrations, then exit
├── api/
│   └── common.yaml          # shared OpenAPI schemas: Error, pagination,
IDs
├── migrations/
│   ├── embed.go             # //go:embed *.sql → migrations.FS
│   └── 00001_platform.sql ... # goose SQL files
├── sqlc.yaml                # one entry per module + platform/events
├── .golangci.yml            # lint config, including module boundary rules
├── internal/
│   ├── platform/            # shared plumbing, no business rules
│   │   ├── config/          # typed config from env
│   │   └── logger/          # slog setup, request ID + actor on log

```

```

records
|  |  | └─ server/           # http.Server, timeouts, graceful shutdown
|  |  | └─ database/        # pgxpool setup, WithTx helper
|  |  | └─ errs/            # typed errors (kind, code, message)
|  |  | └─ web/             # JSON, error → HTTP status, strict-server
error handlers
|  |  | └─ jwt/              # issue and verify access tokens
|  |  | └─ auth/            # Actor in context, Require, bearer middleware
|  |  | └─ middleware/      # request logging (other middleware comes from
chi)
|  |  | └─ events/          # Event, Outbox.Append, Registry, HandleEvent
worker
|  |  | └─ storage/         # save/read uploaded and generated files
|  |  | └─ jobs/            # River client setup, enqueue helper, system
actor
|  | └─ identity/
|  | └─ directory/
|  | └─ inventory/
|  | └─ activity/           # audit trail: subscribes to all events,
serves history
| └─ web/                   # frontend (uses the generated TypeScript
client)
| └─ Makefile               # generate, migrate, run-api, run-worker,
test, lint
└─ go.mod

```

The three binaries share all code under `internal/`; they differ only in which entry points they start.

cmd/migrate

Migrations are embedded in the binary, so deployment needs no separate goose CLI or SQL files on disk. `cmd/migrate` applies the app's goose migrations and River's own tables, then exits. Run it before starting new API or worker versions, as a deploy step or init container. The API never migrates on startup, so several API instances can start at once safely.

```
// migrations/embed.go
package migrations
```

```
import "embed"
```

```
//go:embed *.sql
var FS embed.FS
```

```
// cmd/migrate/main.go (core)
pool, err := database.Open(ctx, cfg.DatabaseURL)
if err != nil { return err }
defer pool.Close()

sqlDB := stdlib.OpenDBFromPool(pool) // goose needs *sql.DB
goose.SetBaseFS(migrations.FS)
if err := goose.SetDialect("postgres"); err != nil { return err }
if err := goose.Up(sqlDB, "."); err != nil { return err }

migrator, err := rivermigrate.New(riverpgxv5.New(pool), nil)
if err != nil { return err }
_, err = migrator.Migrate(ctx, rivermigrate.DirectionUp, nil)
return err
```

During development, `make migrate` runs this binary, and `make migrate-new name=add_assets` creates a new goose file. Down migrations exist for local use only; production only moves forward.

cmd/worker

The worker runs background jobs with River, which stores jobs in the same Postgres database, so there is no extra infrastructure. It builds the same modules as the API, then registers each module's workers instead of mounting routes.

```
// cmd/worker/main.go (core): same modules and registry as the API
registry := events.NewRegistry()
outbox := events.NewOutbox(registry, insertClient)

identityMod := identity.New(pool, tokens, outbox)
directoryMod := directory.New(pool, store, outbox)
inventoryMod := inventory.New(pool, store, outbox, directoryMod.Members,
directoryMod.Locations)
activityMod := activity.New(pool, identityMod.Accounts)
activityMod.Subscribe(registry)

workers := river.NewWorkers()
events.RegisterWorker(workers, pool, registry) // HandleEventWorker: delivers
events to subscribers
inventoryMod.RegisterWorkers(workers)          // CommitImport, ExportAssets

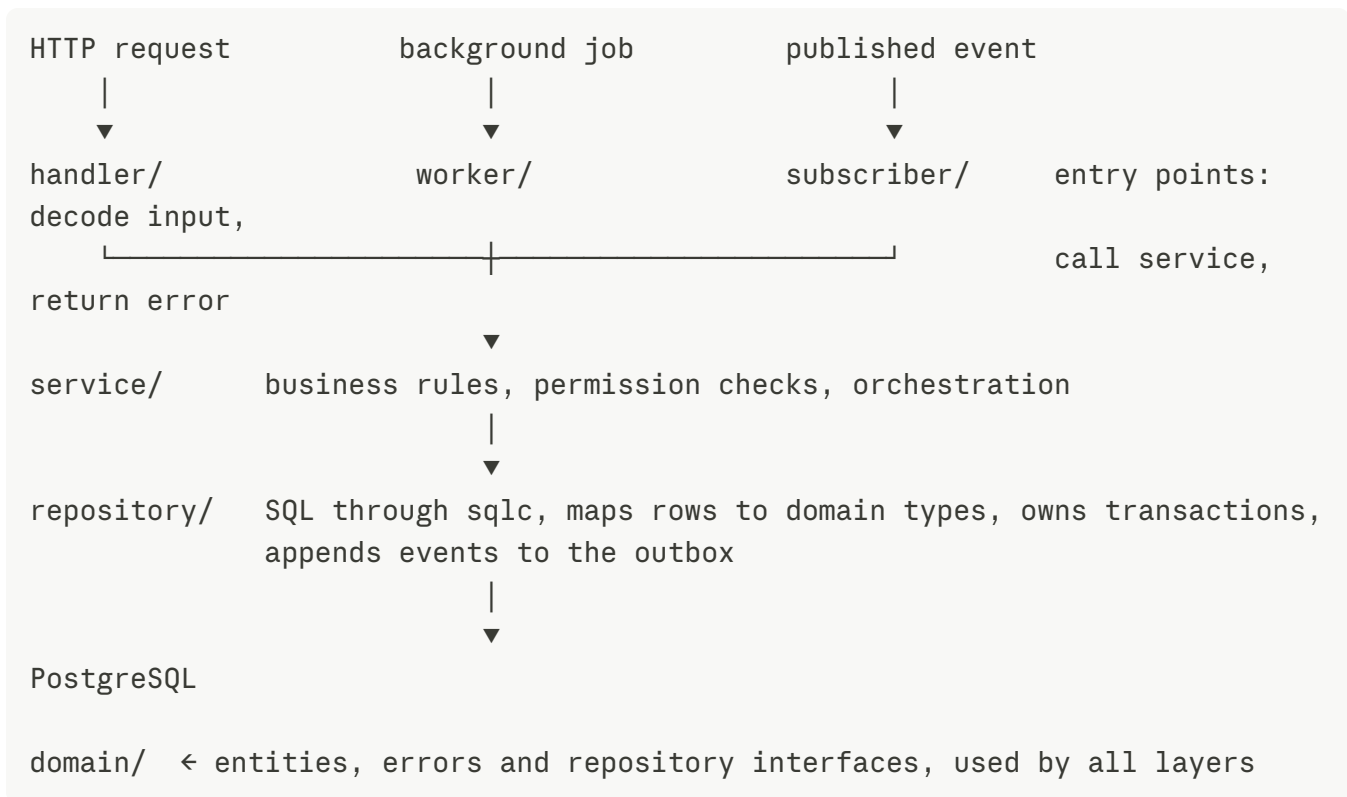
client, err := jobs.NewWorkerClient(pool, workers,
jobs.PeriodicJobs(inventoryMod))
if err != nil { return err }
if err := client.Start(ctx); err != nil { return err }
<-ctx.Done() // SIGINT/SIGTERM
return client.Stop(context.Background())
```

The API uses an insert-only River client (`jobs.NewInsertClient(pool)`) to enqueue jobs, and can enqueue inside a transaction with `InsertTx` so a job exists only if the change that needs it was committed.

Phase 1 jobs: committing large imports (over 1,000 rows) and generating large exports. Scheduled jobs such as overdue-return and warranty reminders use the same worker later.

Architecture rules

Every request follows one path through a module: handler → service → repository → database. Each layer calls only the layer directly below it, and all three share the types in `domain`.



These rules are enforced in code review:

1. Handlers, workers and subscribers are entry points: they call only their own module's service and contain no SQL and no business rules.
2. A service calls its repositories, platform packages, or another module's contract interfaces. It never imports `chi`, `net/http`, `pgx` or `sqlc`.
3. A repository is the only code that imports the generated `sqlc` package. It returns domain types, never `sqlc` structs.
4. `domain` imports only the standard library, `google/uuid` and `platform/errs`.
5. `platform` never imports a module.
6. Permission checks happen in the service, so they apply no matter which handler calls it.
7. `module.go` in each module is the only file that knows all four layers; `cmd/api/main.go` is the only file that knows all modules.

Three boundaries, three contracts

Boundary	Contract	Where it lives
Frontend and API clients ↔ backend	OpenAPI spec, with generated Go server types and TypeScript client	<code><module>/handler/openapi.yaml</code>
Module ↔ module, synchronous	Go interfaces plus plain DTOs	<code><module>/contract/contract.go</code>
Module ↔ module, asynchronous	Published event types and payloads, delivered through the outbox	<code><module>/contract/events.go</code>
Service ↔ database inside a module	Repository interfaces	<code><module>/domain/</code>

Each contract is kept separate from the others. Generated OpenAPI types stay in the handler layer; contract DTOs are not domain entities; domain types never cross a module boundary.

Talking between modules

Each module has a `contract/` package that is its only public surface. It holds interfaces, plain data types and the errors callers may need to check, and nothing else.

```
// directory/contract/contract.go
package contract

type Member struct {
    ID          uuid.UUID
    Name        string
    Email       string
    DepartmentID *uuid.UUID
    Active      bool
}

type MemberReader interface {
    GetMember(ctx context.Context, id uuid.UUID) (Member, error)
}

type LocationReader interface {
```

```

        GetLocation(ctx context.Context, id uuid.UUID) (Location, error)
    }

    var ErrMemberNotFound = errs.New(errs.NotFound, "member_not_found", "member
    not found")

```

The providing module's service implements the interfaces, mapping its domain entities to the DTOs, and `module.go` exposes them. The consuming module's service depends on the interfaces only; `cmd/api/main.go` connects them:

```

// directory/module.go
type Module struct {
    Members    contract.MemberReader
    Locations  contract.LocationReader
    Handler    *handler.Handler
}

// inventory/service/service.go
type Service struct {
    assets     domain.AssetRepository
    members    dircontract.MemberReader
    locations  dircontract.LocationReader
}

// cmd/api/main.go
directoryMod := directory.New(pool)
inventoryMod := inventory.New(pool, store, directoryMod.Members,
directoryMod.Locations)

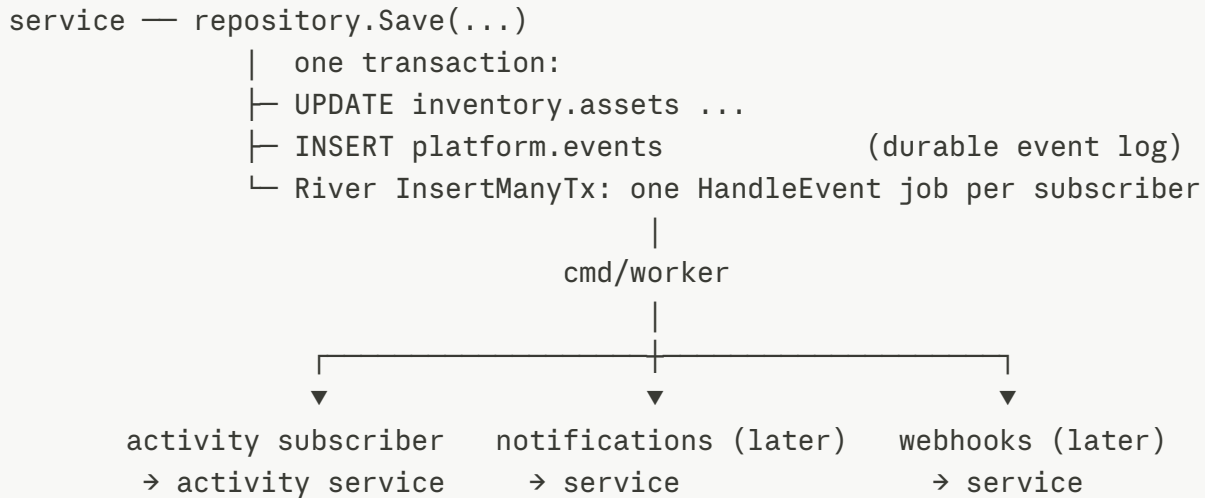
```

Rules for contracts:

- Add a contract method only when another module actually needs it; do not mirror every service method.
- Contracts are small and split by capability (`MemberReader` , not `DirectoryService`), so consumers depend on only what they use and tests need tiny fakes.
- Changing a contract is a deliberate change reviewed by both module owners. Everything else in a module can change freely.
- A module never reads another module's tables and never imports another module's `domain` , `service` , `repository` , `handler` or `worker` . If two changes must happen in one transaction, they belong to the same module.

Events between modules (outbox on River)

When something should happen as a consequence of a change (record history, notify someone, update a cost roll-up), the changing module publishes an event instead of calling the other module. Events use the transactional outbox pattern, with River as the delivery mechanism.



- **Atomic:** the change, the event row and the jobs commit or roll back together, so no event is lost and none is sent for a change that did not happen.
- **Independent retries:** each subscriber has its own River job, so one failing subscriber never blocks or repeats another.
- **At-least-once:** subscribers must be idempotent. River unique jobs prevent duplicate enqueues; subscribers guard re-runs, for example with `ON CONFLICT (event_id) DO NOTHING`.
- **No ordering guarantee:** subscribers must not rely on event order; payloads carry the entity's version when order matters.
- **Durable log:** `platform.events` keeps every event after River cleans up finished jobs, so events can be replayed into a new subscriber.

Published events are part of the publishing module's contract. Entities record internal domain events; the repository maps them to the public types in `contract/events.go` and appends them:

```
// inventory/contract/events.go
const EventAssetCheckedOut = "inventory.asset_checked_out"

type AssetCheckedOut struct {
    AssetID      uuid.UUID `json:"asset_id"`
    Tag          string   `json:"tag"`
    MemberID     uuid.UUID `json:"member_id"`
    ExpectedReturn *time.Time `json:"expected_return,omitempty"`
}
```

```
// platform/events/outbox.go
func (o *Outbox) Append(ctx context.Context, tx pgx.Tx, evts ...Event) error {
    var jobs []river.InsertManyParams
    for _, e := range evts {
        e.ActorID = auth.ActorIDFrom(ctx)
        if err := o.q.WithTx(tx).InsertEvent(ctx, toParams(e)); err != nil {
            return err
        }
        for _, sub := range o.registry.For(e.Type) {
            jobs = append(jobs, river.InsertManyParams{
                Args:      HandleEventArgs{EventID: e.ID,
Subscriber: sub},
                InsertOpts: &river.InsertOpts{UniqueOpts:
river.UniqueOpts{ByArgs: true}},
            })
        }
        if len(jobs) == 0 {
            return nil
        }
        _, err := o.river.InsertManyTx(ctx, tx, jobs)
        return err
    }
}
```

Subscribers are registered in each consuming module's `module.go`. Both `cmd/api` and `cmd/worker` build the same registry: the API needs it to know which jobs to enqueue, the worker to know which code to run.

```
// activity/module.go
func (m *Module) Subscribe(r *events.Registry) {
    r.OnAll("activity.record", m.subscriber.Record)
}

// later, notifications/module.go
func (m *Module) Subscribe(r *events.Registry) {
    r.On(invcontract.EventAssetCheckedOut,
        "notifications.checkout_email", m.subscriber.CheckoutEmail)
}
```

`HandleEventWorker` loads the event, restores the original actor from `ActorID`, and calls the subscriber, so follow-up actions are attributed to the person who made the change. Use a direct contract call instead of an event only when the caller needs an answer now (for example, checking a member exists before a check-out).

Enforcing the boundaries

The rules are checked in CI with `depguard` in `golangci-lint`, so they cannot erode silently. Each module gets a rule that allows only other modules' `contract` packages:

```
# .golangci.yml (excerpt)
linters:
  enable: [depguard]
linters-settings:
  depguard:
    rules:
      inventory-boundaries:
        files: ["**/internal/inventory/**"]
        deny:
          - pkg: "yourapp/internal/directory/domain"
            desc: "use internal/directory/contract"
          - pkg: "yourapp/internal/directory/service"
            desc: "use internal/directory/contract"
          - pkg: "yourapp/internal/directory/repository"
            desc: "use internal/directory/contract"
          - pkg: "yourapp/internal/identity/service"
            desc: "use internal/identity/contract"
      domain-purity:
        files: ["**/internal/*/domain/**"]
```

```
deny:
  - pkg: "github.com/go-chi/chi/v5"
  - pkg: "github.com/jackc/pgx/v5"
  - pkg: "net/http"
```

The same pattern covers the layer rules inside a module (for example, `service` may not import `handler` or the generated `db` package). If the deny lists become long, go-arch-lint expresses the same rules as an allow-list per component.

Module template

Every module has the same four folders plus one wiring file, so anyone who knows one module can navigate all of them.

```
internal/<module>/
├─ contract/
│   └─ contract.go           # public: interfaces, DTOs, errors for other
modules
│   └─ events.go             # public: event type names + payload structs
├─ domain/
│   └─ <entity>.go           # struct, behaviour methods, domain events,
errors, repository interface
│   └─ ...
├─ service/
│   └─ service.go            # Service struct + New(...)
│   └─ <topic>.go            # use-case methods grouped by topic
│   └─ contract.go           # implements contract interfaces (maps domain
→ DTOs)
├─ repository/
│   └─ queries/*.sql          # sqlc input
│   └─ db/                   # sqlc output (generated, committed)
│   └─ <entity>_repository.go # implements the domain interface, appends
events
├─ handler/
│   └─ openapi.yaml           # this module's HTTP contract
│   └─ oapi.yaml              # oapi-codegen config
│   └─ api/api.gen.go         # generated: types, StrictServerInterface, chi
routes
│   └─ handler.go             # Handler struct, New, Mount
│   └─ <resource>.go           # implements the generated interface methods
├─ worker/                   # only if the module has background jobs
│   └─ <job>.go                # job args (Kind) + River worker calling the
```

```

service
├── subscriber/                # only if the module reacts to other modules'
events
|   └─ <reaction>.go         # decode event payload, call the service
└─ module.go                 # New(deps) *Module; exposes contracts,
Handler,

                                # RegisterWorkers, Subscribe

```

What goes where

Folder	Contains	Must not contain
contract	Interfaces other modules may call, plain DTO structs, errors callers check, published event types and payloads	Domain entities, implementations, anything from other layers
domain	Entity structs, behaviour methods that enforce rules (<code>Asset.CheckOut</code>) and record domain events, error values, repository interfaces	SQL, HTTP, JSON tags, sqlc or OpenAPI types
service	One <code>Service</code> struct per module with a method per use case; permission checks; calls to repositories and other modules' contracts; contract implementations	chi, net/http, River, sqlc, generated API types
repository	sqlc queries, repository structs, row ↔ domain mapping, transactions, mapping domain events to contract events and calling <code>Outbox.Append</code>	Business decisions
handler	OpenAPI spec, generated code, methods implementing the generated interface, API ↔ service type mapping	Business rules, database access
worker	River job args and workers: set the actor on the context, call one service method, return its error	Business rules, database access

Folder	Contains	Must not contain
<code>subscriber</code>	One function per reaction: decode another module's event payload (from its <code>contract</code>), call one service method, return its error; must be idempotent	Business rules, database access
<code>module.go</code>	Constructing repository, service, handler, workers and subscribers; exposing contracts; registering subscriptions	Anything else

Business rules live on the domain entity when they concern one entity (can this asset be checked out?) and in the service when they coordinate several things (does the member exist, is the actor allowed, then check out).

Example: identity login, layer by layer

`domain/account.go` :

```
type Account struct {
    ID          uuid.UUID
    Email       string
    Name        string
    PasswordHash string
    Active      bool
    Permissions []string
}

func (a *Account) CanLogin() error {
    if !a.Active {
        return ErrAccountDisabled
    }
    return nil
}

var (
    ErrAccountNotFound = errs.New(errs.NotFound, "account_not_found",
    "account not found")
    ErrInvalidCredentials = errs.New(errs.Unauthenticated,
    "invalid_credentials", "invalid email or password")
)
```

```

        ErrAccountDisabled    = errs.New(errs.Forbidden, "account_disabled",
"account is disabled")
    )

type AccountRepository interface {
    FindByEmail(ctx context.Context, email string) (*Account, error)
    FindByID(ctx context.Context, id uuid.UUID) (*Account, error)
}

```

service/service.go:

```

type Service struct {
    accounts domain.AccountRepository
    jwt       *jwt.Manager
}

func New(accounts domain.AccountRepository, jwt *jwt.Manager) *Service {
    return &Service{accounts: accounts, jwt: jwt}
}

func (s *Service) Login(ctx context.Context, email, password string)
(*LoginResult, error) {
    acc, err := s.accounts.FindByEmail(ctx,
strings.ToLower(strings.TrimSpace(email)))
    if errors.Is(err, domain.ErrAccountNotFound) {
        return nil, domain.ErrInvalidCredentials
    }
    if err != nil {
        return nil, err
    }
    if bcrypt.CompareHashAndPassword([]byte(acc.PasswordHash),
[]byte(password)) != nil {
        return nil, domain.ErrInvalidCredentials
    }
    if err := acc.CanLogin(); err != nil {
        return nil, err
    }
    token, exp, err := s.jwt.Issue(acc.ID, acc.Permissions)
    if err != nil {

```

```

        return nil, err
    }
    return &LoginResult{AccessToken: token, ExpiresAt: exp}, nil
}

```

`repository/account_repository.go`:

```

type AccountRepository struct{ q *db.Queries }

func NewAccountRepository(pool *pgxpool.Pool) *AccountRepository {
    return &AccountRepository{q: db.New(pool)}
}

func (r *AccountRepository) FindByEmail(ctx context.Context, email string)
(*domain.Account, error) {
    row, err := r.q.GetAccountByEmail(ctx, email)
    if errors.Is(err, pgx.ErrNoRows) {
        return nil, domain.ErrAccountNotFound
    }
    if err != nil {
        return nil, err
    }
    perms, err := r.q.ListAccountPermissions(ctx, row.ID)
    if err != nil {
        return nil, err
    }
    return &domain.Account{ID: row.ID, Email: row.Email, Name: row.Name,
        PasswordHash: row.PasswordHash, Active: row.Active,
        Permissions: perms}, nil
}

```

`handler/auth.go` (implements the generated strict interface):

```
func (h *Handler) Login(ctx context.Context, req api.LoginRequestObject)
(api.LoginResponseObject, error) {
    res, err := h.svc.Login(ctx, string(req.Body.Email),
req.Body.Password)
    if err != nil {
        return nil, err // web.Error maps it to status + JSON
    }
    return api.Login200JSONResponse{AccessToken: res.AccessToken,
ExpiresAt: res.ExpiresAt}, nil
}

func (h *Handler) Mount(r chi.Router) {
    strict := api.NewStrictHandlerWithOptions(h, nil,
api.StrictHTTPServerOptions{
        RequestErrorHandlerFunc: web.RequestError,
        ResponseErrorHandlerFunc: web.Error,
    })
    api.HandlerFromMux(strict, r)
}
```

module.go:

```
type Module struct {
    Accounts contract.AccountReader // for modules that show who did what
    Handler  *handler.Handler
}

func New(pool *pgxpool.Pool, jwt *jwt.Manager) *Module {
    svc := service.New(repository.NewAccountRepository(pool), jwt)
    return &Module{Accounts: svc, Handler: handler.New(svc)}
}
```

Platform packages

`internal/platform` holds technical plumbing that every module uses; if a package needs to know what an asset or a member is, it does not belong here.

Package	Responsibility
<code>config</code>	One <code>Config</code> struct loaded from environment variables at startup; fail fast on missing values

Package	Responsibility
logger	slog JSON handler setup; logger.With(ctx) adds request ID and actor ID to log records
server	http.Server with read, write and idle timeouts; Run(ctx) shuts down gracefully on SIGINT/SIGTERM
database	Open(ctx, url) creates *pgxpool.Pool; WithTx(ctx, pool, func(pgx.Tx) error) error commits or rolls back
errs	Typed errors: New(kind, code, message) with kinds Invalid, NotFound, Conflict, Unauthenticated, Forbidden
web	JSON writer; Error(w, r, err) maps errors to status + JSON and is the strict server's response error handler; RequestError handles malformed requests; Handle for the few plain handlers outside OpenAPI (for example the /a/{tag} redirect)
jwt	Manager with Issue(accountID, permissions) and Verify(token); HS256, issuer check, 15-minute expiry
auth	Actor (account ID, permissions), WithActor / FromContext, Require(ctx, permission), SystemActor for scheduled jobs, and the bearer-token Middleware
middleware	RequestLogger : logs method, path, status, duration, request ID and actor via logger; generic middleware (RequestID, RealIP, Recoverer, Timeout) is used straight from go-chi/chi/v5/middleware

Package	Responsibility
<code>events</code>	<code>Event</code> envelope; <code>Outbox.Append(ctx, tx, events...)</code> writes event rows and one River job per subscriber in the caller's transaction; <code>Registry</code> maps event types to named subscribers; <code>HandleEventWorker</code> delivers an event to one subscriber
<code>storage</code>	<code>Store</code> interface (<code>Put</code> , <code>Get</code> , <code>Delete</code>) with a local-disk implementation for development and an S3-compatible one for production; keeps uploaded import files, error reports and large exports
<code>jobs</code>	<code>NewInsertClient(pool)</code> for the API, <code>NewWorkerClient(pool, workers, periodic)</code> for the worker, queue names, retry defaults, and <code>ActorArgs</code> so a job carries the ID of the account that requested it

`events` and `jobs` both sit on River: `jobs` is for work a module asks for explicitly (commit a large import), `events` is for announcing that something happened so any number of subscribers can react. The event design is described under Architecture rules.

Jobs run as the account that requested them: the worker loads that account's permissions and puts an `Actor` on the context before calling the service, so permission checks and the events they append behave exactly as in the API. Scheduled jobs use `auth.SystemActor`.

Handlers return errors

Handlers have the signature `func(w, r) error`. `web.Handle` wraps them for chi and turns any returned error into a response in one place:

```
type HandlerFunc func(w http.ResponseWriter, r *http.Request) error

func Handle(h HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if err := h(w, r); err != nil {
            Error(w, r, err)
        }
    }
}
```

```
}

func Error(w http.ResponseWriter, r *http.Request, err error) {
    var e *errs.Error
    if errors.As(err, &e) {
        _ = JSON(w, statusByKind[e.Kind], errorBody{Code: e.Code,
Message: e.Message})
        return
    }
    slog.ErrorContext(r.Context(), "internal error", "err", err,
        "request_id", middleware.GetReqID(r.Context()))
    _ = JSON(w, http.StatusInternalServerError, errorBody{Code:
"internal", Message: "internal error"})
}
```

Authentication middleware

The middleware reads the bearer token, verifies it with `platform/jwt`, and stores the actor in the request context. Services then call `auth.Require`.

```

func Middleware(tokens *jwt.Manager) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return web.Handle(func(w http.ResponseWriter, r
*http.Request) error {
            raw, ok :=
strings.CutPrefix(r.Header.Get("Authorization"), "Bearer ")
            if !ok || raw == "" {
                return errs.New(errs.Unauthenticated,
"missing_token", "missing bearer token")
            }
            claims, err := tokens.Verify(raw)
            if err != nil {
                return errs.New(errs.Unauthenticated,
"invalid_token", "token is invalid or expired")
            }
            actor := Actor{AccountID: claims.AccountID,
Permissions: claims.Permissions}
            next.ServeHTTP(w,
r.WithContext(WithActor(r.Context(), actor)))
            return nil
        })
    }
}

```

Permission constants are declared by the module that owns them (for example `inventory/domain.PermAssetCreate`).

Identity module

Identity owns who can sign in and what they may do: accounts, passwords, roles and permissions, and issuing JWTs. It does not own people; a person's name, department and assets live in directory, and an account links to at most one member.

```

internal/identity/
├─ contract/
│   └─ contract.go           # AccountReader (name/email for "changed by"),
Account DTO
│   └─ events.go             # account_created, account_disabled,
roles_assigned, ...
├─ domain/
│   └─ account.go            # Account, CanLogin, errors, AccountRepository

```

```

|   ├── role.go           # Role, RoleRepository
|   └── permissions.go    # identity permission constants
└── service/
    ├── service.go        # Service, New
    ├── auth.go           # Login, Me
    ├── accounts.go       # CreateAccount, DisableAccount, AssignRoles,
SetPassword
    ├── roles.go          # ListRoles, CreateRole, UpdateRolePermissions
    └── contract.go       # implements contract.AccountReader
└── repository/
    ├── queries/          # accounts.sql, roles.sql
    ├── db/               # sqlc output
    ├── account_repository.go
    └── role_repository.go
└── handler/
    ├── openapi.yaml      # /auth/login, /me, /accounts/*, /roles/*
    ├── oapi.yaml
    ├── api/api.gen.go
    ├── handler.go        # Handler, Mount
    ├── auth.go           # Login, GetMe
    ├── accounts.go
    └── roles.go
└── module.go

```

RBAC

Permissions are strings named `<module>.<resource>.<action>`, such as `inventory.asset.create`, `inventory.asset.import`, `directory.member.update` and `identity.role.manage`. Each module declares its own constants in its `domain` package.

Roles are named sets of permissions stored in the database. An account has one or more roles, and its effective permissions are the union. Seed these system roles in a migration; they can be edited but not deleted:

Role	Intended for	Permissions (summary)
Admin	IT lead, system owner	Everything, including role and account management
Asset Manager	IT staff	All inventory permissions, directory read/write, no role management

Role	Intended for	Permissions (summary)
Viewer	Auditors, finance	Read-only across inventory and directory, export allowed
Member	Every employee	Read own profile and assets assigned to them

Services check permissions first: `actor, err := auth.Require(ctx, domain.PermAssetCreate)`. Checks based on data, such as members seeing only their own assets, also live in the service.

Authentication flow

1. `POST /api/v1/auth/login` calls `Service.Login`: look up the account by lowercased email, compare with bcrypt, and return the same `invalid_credentials` error for an unknown email and a wrong password.
2. On success, `platform/jwt` issues an HS256 token with `sub` = account ID, `iss` = app name, `exp` = 15 minutes, and a `perms` claim with the account's permissions.
3. The client sends `Authorization: Bearer <token>`. `auth.Middleware` verifies signature, algorithm, issuer and expiry, then puts an `Actor` in the request context.
4. Because permissions travel in the token, role changes apply when the token expires. Keep the expiry short.
5. The signing secret comes from the `JWT_SECRET` environment variable and must be at least 32 random bytes.

Admins create accounts with `CreateAccount`, which stores a bcrypt hash of an initial password or sends a set-password link. Refresh tokens, logout after failed logins, SSO and MFA are later additions; each fits inside identity without changing the other modules.

Directory module

Directory owns the company's people and places: members, departments and locations. Most members never sign in; those who do have an identity account pointing at their member record.

```
internal/directory/
├── contract/
│   ├── contract.go          # MemberReader, LocationReader, Member/Location
└── DT0s, errors
```

```

|   └─ events.go           # member_created, member_updated,
member_offboarded, ...
└─ domain/
|   └─ member.go           # Member, Offboard, errors, MemberRepository
|   └─ department.go       # Department, DepartmentRepository
|   └─ location.go         # Location (site/building/floor/room),
LocationRepository
|   └─ permissions.go
└─ service/
|   └─ service.go
|   └─ members.go         # GetMember, ListMembers, CreateMember,
UpdateMember, OffboardMember
|   └─ departments.go      # department CRUD + tree
|   └─ locations.go        # location CRUD + tree
|   └─ import.go           # member import/export from Excel/CSV
|   └─ contract.go         # implements MemberReader, LocationReader
└─ repository/
|   └─ queries/            # members.sql, departments.sql, locations.sql
|   └─ db/
|   └─ *_repository.go
└─ handler/               # openapi.yaml (/members, /departments,
/locations), api/, handlers
└─ module.go

```

Rules

- Members are never hard-deleted. `OffboardMember` sets status to `offboarded` and records the date. The offboarding page asks inventory's contract for assets still assigned to that member.
- Employee number and email are unique among active members; enforce this with partial unique indexes and translate violations into an `errs.Conflict` error in the repository.
- Departments and locations are trees stored with a `parent_id`. The service rejects moves that would create a cycle; reads use recursive CTEs.
- Location kinds follow a fixed order (site, building, floor, room); `Location.CanHaveParent` enforces the allowed parent kinds.
- A manager must be an active member and cannot be the member themselves.

What other modules use

Directory publishes `MemberReader` and `LocationReader` in its `contract` package. Inventory's service depends on those interfaces to validate check-outs and moves, and directory's offboarding page uses inventory's `contract.AssetsByMember` to list assets still assigned. Neither module imports the other's service, repository or tables.

The "employee view" page is two calls from the frontend: directory's `GET /members/{memberID}` and inventory's `GET /members/{memberID}/assets`. The second route lives in inventory because inventory owns custody.

Vendors are a plain text field on assets in phase 1. When maintenance and contracts arrive, promote them to a `Vendor` entity in directory.

Inventory module

Inventory owns assets and everything that must stay consistent with them: categories, custom fields, statuses, tags, parent/child links, custody (check-in/out), and spreadsheet import and export.

```
internal/inventory/
├── contract/
│   ├── contract.go          # AssetsByMember (for offboarding), AssetSummary
│   └── DTO
│       ├── events.go        # asset_created, asset_updated, asset_checked_out,
│       └── ...
├── domain/
│   ├── asset.go             # Asset, CheckOut, CheckIn, ChangeStatus,
│   ├── AttachTo, errors, AssetRepository
│   ├── category.go          # Category, FieldDefinition, ValidateCustomFields,
│   ├── CategoryRepository
│   ├── status.go            # Status, StatusKind, StatusRepository
│   ├── import.go            # Import, ImportRow, ImportRepository
│   └── permissions.go
├── service/
│   ├── service.go           # Service, New (takes directory's MemberReader,
│   ├── LocationReader)
│   ├── assets.go            # CreateAsset, UpdateAsset, GetAsset, ListAssets,
│   └── AssetTree
│       ├── custody.go        # CheckOut, CheckIn, AssetsByMember
│       ├── categories.go
│       └── statuses.go
```

```

|   ├── import.go          # UploadImport, CommitImport
|   ├── export.go         # ExportAssets
|   └── contract.go       # implements contract.AssetsByMember
└── repository/
    ├── queries/          # assets.sql, categories.sql, statuses.sql,
imports.sql
    ├── db/
    └── *_repository.go
└── spreadsheet/         # read/write xlsx (excelize) and csv rows; import
template
    ├── handler/
    │   ├── openapi.yaml  # /assets, /categories, /statuses, /imports,
/members/{memberID}/assets
    │   ├── oapi.yaml
    │   ├── api/api.gen.go
    │   ├── handler.go    # Mount, plus the plain /a/{tag} redirect
    │   └── *.go          # generated-interface methods by resource
    └── worker/
        ├── commit_import.go # CommitImportArgs + worker
        └── module.go

```

`spreadsheet/` is a small helper package used only by the inventory service (and later by directory for member import). It knows about rows and cells, never about assets.

Asset entity

An asset holds identity (ID, tag, name, serial number, make, model), classification (category ID, status ID), placement (location ID, parent asset ID), custody (holder member ID, checked-out at, expected return), purchase data (date, cost in minor units plus currency code, vendor text, order number), warranty expiry, custom field values, and a version number for optimistic locking.

Rules that concern one asset are methods on it:

- `CheckOut(memberID, expectedReturn, inUseStatus, now)` requires a status of kind `available`, no current holder, and no parent asset (children move with their parent).
- `CheckIn(availableStatus, locationID, now)` clears custody and optionally sets a new location.
- `ChangeStatus(status)` rejects moving an asset with a holder into an `archived` status.

- `AttachTo(parentID)` rejects self-reference; the service also asks the repository whether the parent is a descendant, to prevent cycles.

The `CheckOut` service method shows how the layers work together: check the permission, call `members.GetMember` (directory's `MemberReader` contract) to confirm the member is active, load the asset, call `asset.CheckOut`, then `repo.SaveCheckOut`, which updates the asset and appends an `asset_checked_out` event in one transaction.

Categories and custom fields

A category defines its custom fields: key, label, type (text, number, date, boolean, select), required flag, and options for select. Values are stored in an `assets.custom_fields` JSONB column and validated by `Category.ValidateCustomFields` before saving. Removing a field definition hides the value but does not delete stored data.

Statuses

Statuses are rows the company can add, rename and reorder. Each maps to a fixed kind that the code relies on:

Kind	Meaning	Example statuses
<code>pending</code>	Not yet received	Ordered
<code>available</code>	Can be checked out	In stock
<code>in_use</code>	Assigned to someone	Deployed
<code>unavailable</code>	Temporarily out of service	In repair, Lost/stolen
<code>archived</code>	End of life	Retired, Disposed

Settings mark one default status for `available` and one for `in_use`; check-in and check-out use those.

Tags and QR codes

Asset tags are generated in the insert query from a Postgres sequence, formatted as a prefix plus a zero-padded number (for example `AST-000123`). Imported rows may carry an existing tag, which is accepted if unique. QR codes encode `https://<app>/a/{tag}`; the route redirects to the asset page, so any phone camera works.

Excel and CSV import

Import has two steps, so nothing is written until the user has seen the result.

1. **Upload and validate.** `POST /imports` (multipart, max 10 MB, 10,000 rows) reads rows with `spreadsheet`, maps headers to fields (custom fields by label), loads categories, statuses, locations and members once into maps, and checks each row with the same domain rules as manual entry. It saves an `Import` with per-row results (`create`, `update`, `error` with messages) and returns a summary.
2. **Commit.** `POST /imports/{importID}/commit` applies valid rows in transactions of 500 rows, marks each applied, and appends events. Re-running skips applied rows, so it is safe to retry.

`GET /imports/{importID}/errors.xlsx` returns the uploaded file with an error column added, so users fix it in Excel and re-upload. In upsert mode, rows match existing assets by tag, then by serial number. `GET /assets/import-template.xlsx` returns headers plus dropdowns for category and status.

Imports of up to 1,000 rows commit inside the request. Larger commits are enqueued as a `CommitImport` job in `inventory/worker/`, which calls the same `Service.CommitImport` method; the client polls `GET /imports/{importID}` for progress.

Export

Export uses the same columns as import, so a user can export, edit in Excel, and re-import. `GET /assets/export.xlsx` accepts the same filters as the asset list and streams rows with excelize's stream writer.

Activity module

Activity is the audit trail: it subscribes to every published event and turns it into an append-only history entry, so any change in any module shows who did what and when without that module writing history itself.

```

internal/activity/
├── domain/
│   └── entry.go          # Entry, EntryRepository
├── service/
│   ├── service.go
│   ├── record.go        # Record(ctx, event): event → entry, idempotent
│   └── history.go        # History(ctx, entityType, entityID, page)
├── repository/
│   ├── queries/entries.sql
│   ├── db/
│   └── entry_repository.go
├── subscriber/
│   └── record.go          # registered with
registry.OnAll("activity.record", ...)
├── handler/              # openapi.yaml: GET
/history/{entityType}/{entityID}
└── module.go              # New(pool, identity AccountReader), Subscribe,
Handler

```

- `activity.entries` stores event ID (unique), entity type and ID, action, actor ID, occurred at, and a JSON summary. For update events, the payload's changed fields with before and after values are kept.
- Recording is idempotent: `INSERT ... ON CONFLICT (event_id) DO NOTHING`, so River retries never duplicate entries.
- History is eventually consistent: an entry appears within seconds of the change. It is still guaranteed, because the job commits in the same transaction as the change.
- `History` resolves actor names through identity's `AccountReader` contract and checks read permission for the entity type (for example `inventory.asset.read` for assets).
- Entries are never updated or deleted. Activity has no events of its own and no module depends on it, so it can be rebuilt by replaying `platform.events`.

Database and sqlc conventions

One migrations folder is the single source of schema, each module owns a Postgres schema of the same name, and sqlc generates one `db` package per module inside its `repository/` folder.

Schema

- Postgres schemas: `platform` (events), `identity`, `directory`, `inventory`, `activity`; River keeps its tables in the default schema. A module writes only to its own schema; `platform/events` writes to `platform`.
- Migrations are goose SQL files named `NNNNN_<module>_<change>.sql` with `-- +goose Up / -- +goose Down` sections. Never edit a migration that has been applied anywhere; add a new one.
- Primary keys are `uuid`, generated in Go with `uuid.NewV7()` so they sort by creation time. Event IDs are generated the same way.
- Every table has `created_at` and `updated_at` as `timestampz not null`.
- Main entities (assets, members, accounts) have a `version int not null` column for optimistic locking.
- Money is `bigint` minor units plus a `char(3)` currency code. No `numeric` or floats for money.
- Records people care about are never hard-deleted; use a status or `archived_at`.
- Foreign keys across schemas are allowed (for example `inventory.assets.location_id` references `directory.locations`).

sqlc.yaml

```
version: "2"
sql:
  - engine: postgresql
    schema: migrations
    queries: internal/identity/repository/queries
    gen:
      go:
        package: db
        out: internal/identity/repository/db
        sql_package: pgx/v5
        emit_empty_slices: true
        overrides:
          - db_type: uuid
            go_type: github.com/google/uuid.UUID
          - db_type: jsonb
            go_type: encoding/json.RawMessage
# repeat for directory and inventory
```

Query files

- One `.sql` file per table in `repository/queries/`.
- Names say what the query does: `GetAssetByID`, `GetAssetForUpdate` (with `FOR UPDATE`), `ListAssets`, `CountAssets`, `InsertAsset`, `UpdateAsset`.
- Optional filters use `sqlc.narg: (sqlc.narg('status_id')::uuid IS NULL OR status_id = sqlc.narg('status_id'))`.
- Lists of IDs use `= ANY(@ids::uuid[])`.
- Updates check the version: `UPDATE ... SET ..., version = version + 1 WHERE id = @id AND version = @version`. Zero rows affected becomes a `Conflict` error.
- Bulk import inserts can use `:copyfrom`; trees use recursive CTEs with a depth limit.

Repositories and transactions

A repository method that writes does everything for that change in one transaction, including appending its events to the outbox. The service never sees a transaction.

```
func (r *AssetRepository) SaveCheckOut(ctx context.Context, a *domain.Asset)
error {
    return database.WithTx(ctx, r.pool, func(tx pgx.Tx) error {
        q := r.q.WithTx(tx)
        n, err := q.UpdateAssetCustody(ctx,
db.UpdateAssetCustodyParams{
            ID: a.ID, HolderID: a.HolderID, StatusID: a.StatusID,
            ExpectedReturn: a.ExpectedReturn, Version: a.Version,
        })
        if err != nil {
            return err
        }
        if n == 0 {
            return domain.ErrAssetChanged // someone else updated
it first
        }
        return r.outbox.Append(ctx, tx,
toContractEvents(a.PullEvents())...)
    })
}
```

Generated types (`db.*`, `pgtype.*`) never leave `repository/`. Each repository translates `pgx.ErrNoRows` to its domain's not-found error and unique violations (SQLSTATE 23505) to a `Conflict` error.

Tooling

`make generate` runs `sqlc generate`; generated code is committed. CI runs `sqlc diff` (fails on stale code) and `sqlc vet`, then applies all migrations to an empty database before tests.

HTTP, OpenAPI and chi conventions

The HTTP API is spec-first: each module's `handler/openapi.yaml` is written before the code, oapi-codegen generates the types and a strict server interface from it, and the handler implements that interface. If the code drifts from the spec, it does not compile.

```
// cmd/api/main.go (core)
tokens := jwt.New(cfg.JWTSecret, "assetapp", 15*time.Minute)

registry := events.NewRegistry()
insertClient, err := jobs.NewInsertClient(pool)
if err != nil { return err }
outbox := events.NewOutbox(registry, insertClient)

identityMod := identity.New(pool, tokens, outbox)
directoryMod := directory.New(pool, store, outbox)
inventoryMod := inventory.New(pool, store, outbox, directoryMod.Members,
directoryMod.Locations)
activityMod := activity.New(pool, identityMod.Accounts)

// Subscriptions are registered in the API too, so Append knows which jobs to
enqueue.
activityMod.Subscribe(registry)

r := chi.NewRouter()
r.Use(chimw.RequestID, chimw.RealIP, middleware.RequestLogger(log),
chimw.Recoverer)
r.Use(chimw.Timeout(30 * time.Second))

r.Route("/api/v1", func(r chi.Router) {
    r.Use(auth.Middleware(tokens, auth.Public("POST
```

```

/api/v1/auth/login"))
    identityMod.Handler.Mount(r)
    directoryMod.Handler.Mount(r)
    inventoryMod.Handler.Mount(r)
    activityMod.Handler.Mount(r)
})
r.Get("/a/{tag}", web.Handle(inventoryMod.Handler.RedirectByTag))

return server.New(cfg.Addr, r).Run(ctx)

```

Because each generated router registers all of a module's paths at once, public operations are listed in `auth.Middleware` instead of being mounted in a separate group.

Writing the spec

```

# inventory/handler/openapi.yaml (excerpt)
openapi: 3.0.3
info: { title: Inventory API, version: 1.0.0 }
paths:
  /assets/{assetID}/check-out:
    post:
      operationId: CheckOutAsset
      parameters:
        - { name: assetID, in: path, required: true, schema: { type: string,
format: uuid } }
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CheckOutRequest' }
      responses:
        '204': { description: Checked out }
        default:
          description: Error
          content:
            application/json:
              schema: { $ref:
'../../api/common.yaml#/components/schemas/Error' }
components:
  schemas:
    CheckOutRequest:
      type: object

```

```

    required: [member_id]
  properties:
    member_id: { type: string, format: uuid }
    expected_return: { type: string, format: date-time }

```

```

# inventory/handler/oapi.yaml
package: api
output: api/api.gen.go
generate:
  models: true
  chi-server: true
  strict-server: true
import-mapping:
  ../../../../api/common.yaml: yourapp/internal/platform/web/apicommon

```

Shared schemas (`Error` , pagination, ID formats) live in `api/common.yaml` , generated once into `platform/web/apicommon` and referenced by every module spec through `import-mapping` . Each module keeps its own spec, so no module has to implement another module's operations.

Handler shape

Each method of the generated `StrictServerInterface` receives a typed request object and returns a typed response or an error. It maps API types to service arguments, calls one service method, and maps the result back.

```

func (h *Handler) CheckOutAsset(ctx context.Context, req
api.CheckOutAssetRequestObject) (api.CheckOutAssetResponseObject, error) {
    err := h.svc.CheckOut(ctx, req.AssetId, req.Body.MemberId,
req.Body.ExpectedReturn)
    if err != nil {
        return nil, err
    }
    return api.CheckOutAsset204Response{}, nil
}

func (h *Handler) Mount(r chi.Router) {
    strict := api.NewStrictHandlerWithOptions(h, nil,
api.StrictHTTPServerOptions{
        RequestErrorHandlerFunc: web.RequestError, // malformed
JSON, bad path params → 422
        ResponseErrorHandlerFunc: web.Error,      // errs kinds →
status codes, others → 500
    })
    api.HandlerFromMux(strict, r)
}

```

The error rule is unchanged: handlers return errors, and `web.Error` alone decides the status code and JSON body. Only document error responses in the spec as `default` with the shared `Error` schema, so handlers never build error responses by hand.

Generation workflow

- Each `handler/handler.go` carries `//go:generate go tool oapi-codegen -config oapi.yaml openapi.yaml`, with `oapi-codegen v2` declared as a tool dependency in `go.mod`.
- `make generate` runs `go generate ./...` (OpenAPI) and `sqlc generate`; generated code is committed, and CI fails if regenerating changes anything.
- The frontend generates its TypeScript client from the same specs (for example `npx openapi-typescript internal/inventory/handler/openapi.yaml -o web/src/api/inventory.ts`).
- Optionally, add `oapi-codegen`'s request validation middleware (built on `kin-openapi`) per module, so formats, required fields and enums are rejected before the handler runs.

API conventions

- Resource paths are plural nouns; actions that are not plain updates are `POST` sub-paths: `/assets/{assetID}/check-out`, `/imports/{importID}/commit`.
- `operationId` is PascalCase and matches the service method where possible (`CheckoutAsset` → `Service.CheckOut`).
- JSON fields are `snake_case`. Generated API types never leave the handler layer, and domain entities are never serialised directly.
- Lists accept `page` and `page_size` (default 50, max 200) and return `{"items": [...], "total": 123}`, using the shared pagination schema.
- Errors always have the shape `{"error": {"code": "asset_not_available", "message": "..."} }`.
- Error kinds map to status codes: `Invalid` 422, `NotFound` 404, `Conflict` 409, `Unauthenticated` 401, `Forbidden` 403; any other error is 500 and logged with the request ID.
- File uploads are `multipart/form-data` operations in the spec with an explicit size limit; downloads declare their media type (for example `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`).

Errors, audit trail and testing

Errors

Each module declares its errors once in `domain`, with a stable code, using `platform/errs`. Services and repositories return them unchanged or wrapped with `fmt.Errorf("...: %w", err)`; handlers just return them, and only `web.Error` decides the status code.

```
// inventory/domain/asset.go
var (
    ErrAssetNotFound      = errs.New(errs.NotFound, "asset_not_found",
"asset not found")
    ErrAssetNotAvailable = errs.New(errs.Conflict, "asset_not_available",
"asset is not available for check-out")
    ErrAssetChanged       = errs.New(errs.Conflict, "asset_changed",
"asset was changed by someone else; reload and retry")
    ErrCyclicParent       = errs.New(errs.Invalid, "cyclic_parent", "asset
cannot be its own ancestor")
)
```

Never return raw database errors from a repository when a domain error fits, and never put SQL or stack traces in a message sent to clients.

Audit trail

The audit trail is built from events. Every repository method that changes data appends its events to the outbox inside its transaction, so a change cannot be saved without its events, and the `activity` module records every event as a history entry (see Activity module). Update events carry the changed fields with before and after values, so edits show exactly what changed.

History is read through activity's `GET /history/{entityType}/{entityID}`. Entries are append-only, and the full event log in `platform.events` can rebuild them if the activity schema changes.

Testing

Layer	How to test
<code>domain</code>	Plain table-driven unit tests on entity methods, including which domain events they record
<code>service</code>	Tests with small hand-written fakes for repositories and other modules' contracts, plus a fixed actor in the context; include permission-denied cases

Layer	How to test
repository	Integration tests against real Postgres (testcontainers-go): the change, the event row and the River jobs are all present after commit and all absent after rollback
contract implementations	A shared test per contract that checks the DTO mapping and not-found errors, so consumers can rely on the behaviour their fakes imitate
subscriber	Call the subscriber twice with the same event and check the effect happens once (idempotency)
spreadsheet	Fixture .xlsx and .csv files in testdata/, including bad headers, empty rows and wrong types
handler	httptest requests against Mount with a real service over fakes; responses checked against the spec's status codes and shapes
Architecture	golangci-lint with depguard rules in CI; a boundary violation fails the build
End to end	A few flows through the full router and worker with a real database: login, create asset, check out, import, and the history entries they produce

The import flow gets the most fixture coverage because it is the first thing a new user touches and the easiest to break with odd spreadsheets.

Naming conventions and feature checklist

Naming

Thing	Convention	Example
Service method	Verb + noun, named after the use case	CheckOut , CreateAsset , OffboardMember

Thing	Convention	Example
OpenAPI operationId	PascalCase verb + noun; becomes the handler method name	<code>CheckOutAsset</code> , <code>ListMembers</code>
Contract interface	Capability + <code>Reader</code> / <code>Writer</code> /verb, small and specific	<code>MemberReader</code> , <code>AssetsByMember</code>
Contract DTO	Plain noun, no methods with business rules	<code>contract.Member</code> , <code>contract.AssetSummary</code>
Repository method	<code>Find...</code> for reads, <code>Save...</code> / <code>Insert...</code> for writes	<code>FindByID</code> , <code>SaveCheckOut</code>
Domain error	<code>Err</code> + description	<code>ErrAssetNotAvailable</code>
Permission	<code><module>.<resource>.<action></code>	<code>inventory.asset.checkout</code>
sqlc query	Verb + noun	<code>GetAssetForUpdate</code> , <code>InsertAsset</code>
Import aliases for contracts	Module prefix	<code>dircontract</code> , <code>invcontract</code>
Tables	Plural snake case in the module schema	<code>inventory.assets</code> , <code>directory.members</code>
Path params	camelCase + <code>ID</code>	<code>{assetID}</code> , <code>{importID}</code>

Avoid package names like `utils` , `common` , `helpers` or `models` . If a helper has no obvious home, it probably belongs on a domain type.

Checklist for adding a feature

1. **Spec:** add or change the operation in the module's `handler/openapi.yaml` and review it with the frontend developer first.
2. **Domain:** add the fields, rule methods, domain events and error values to the entity in `domain/` ; write unit tests.
3. **Permission:** add a constant in `domain/permissions.go` if needed, and grant it to seeded roles in a migration.
4. **Database:** add a goose migration if the schema changes, then the sqlc query.

5. **Generate:** run `make generate` (oapi-codegen + sqlc); the build now fails until the new handler method exists.
6. **Repository:** add the method, map rows to domain types, map domain events to `contract/events.go` types, and call `Outbox.Append` in the same transaction.
7. **Service:** add the use-case method: `auth.Require`, load, call entity methods, save. Use other modules only through their `contract` interfaces. Test it with fakes.
8. **Contract (only if another module needs it):** add the interface method and DTO in `contract/`, or a new event type in `contract/events.go`.
9. **Handler:** implement the generated method: map request → service call → typed response; add an `httptest` test.
10. **Wiring:** only touch `module.go` or `cmd/` if the service needs a new dependency or the module subscribes to a new event.
11. Run `make generate test lint`; CI also checks generated code is current and depguard boundary rules pass.

Roadmap: how later modules fit in

Later features arrive as new modules with the same four folders; none of them should require reshaping identity, directory or inventory.

Later module	Plugs in through	Needs from phase 1
<code>notifications</code>	Subscribes to <code>asset_checked_out</code> and similar events; a scheduled job reads overdue check-outs and warranty expiries through an inventory contract	Published events; <code>expected_return</code> and <code>warranty_expiry</code> columns
<code>webhooks</code>	Subscribes to all events and delivers them to customer URLs, each delivery its own retryable job	Durable event log and stable event payloads
<code>maintenance</code>	Own entities (repair tickets, schedules); changes asset status through an <code>AssetStatusChanger</code> contract; publishes repair cost events	<code>unavailable</code> status kind

Later module	Plugs in through	Needs from phase 1
finance	Subscribes to asset and maintenance cost events to keep cost roll-ups; depreciation as a scheduled job	Money as minor units; purchase data in events
requests	Approval workflow; on approval, calls an inventory <code>AssetCheckOuter</code> contract	Roles and permissions; check-out use case
stocktake	Scan sessions compare scanned tags against expected assets per location via inventory and directory contracts	Tags, QR route, location tree
reporting	Read-only SQL over views that join module schemas (the one documented exception to "no reading other modules' tables")	Stable schemas per module
integrations	Connectors call directory and inventory contracts with external data	Import logic reusable for single records

Scheduled work (reminders, alerts, syncs) registers periodic jobs with the existing `cmd/worker`; each is a `worker/` file in its module calling a service method. Refresh tokens, SSO and MFA extend identity directly without changing how `auth.Middleware` builds the actor.